

Contents

UDP 图传架构说明1

0. 目录1

1. 系统概览1

2. 硬件自适应系统3

3. 解码后端架构5

4. 图传尺寸控制链路6

5. 三缓冲 PBO 工作流程 (OpenGL 路径)7

6. 数据结构定义 (NvdecStub.h)8

7. 配置分级系统8

8. 性能指标9

9. 关键文件清单9

10. 启动流程10

11. 故障排查10

12. 版本历史11

UDP 图传架构说明

最后更新: 2026-02-09 当前状态: 生产就绪 - 支持 2K/4K 120fps, 三缓冲 PBO 优化, 硬件自适应

0. 目录

1. 系统概览

2. 硬件自适应系统 □ 新增

3. 解码后端架构 □ 新增

4. 图传尺寸控制链路

5. 三缓冲 PBO 工作流程

6. 数据结构定义

7. 配置分级系统 □ 新增

8. 性能指标

9. 关键文件清单

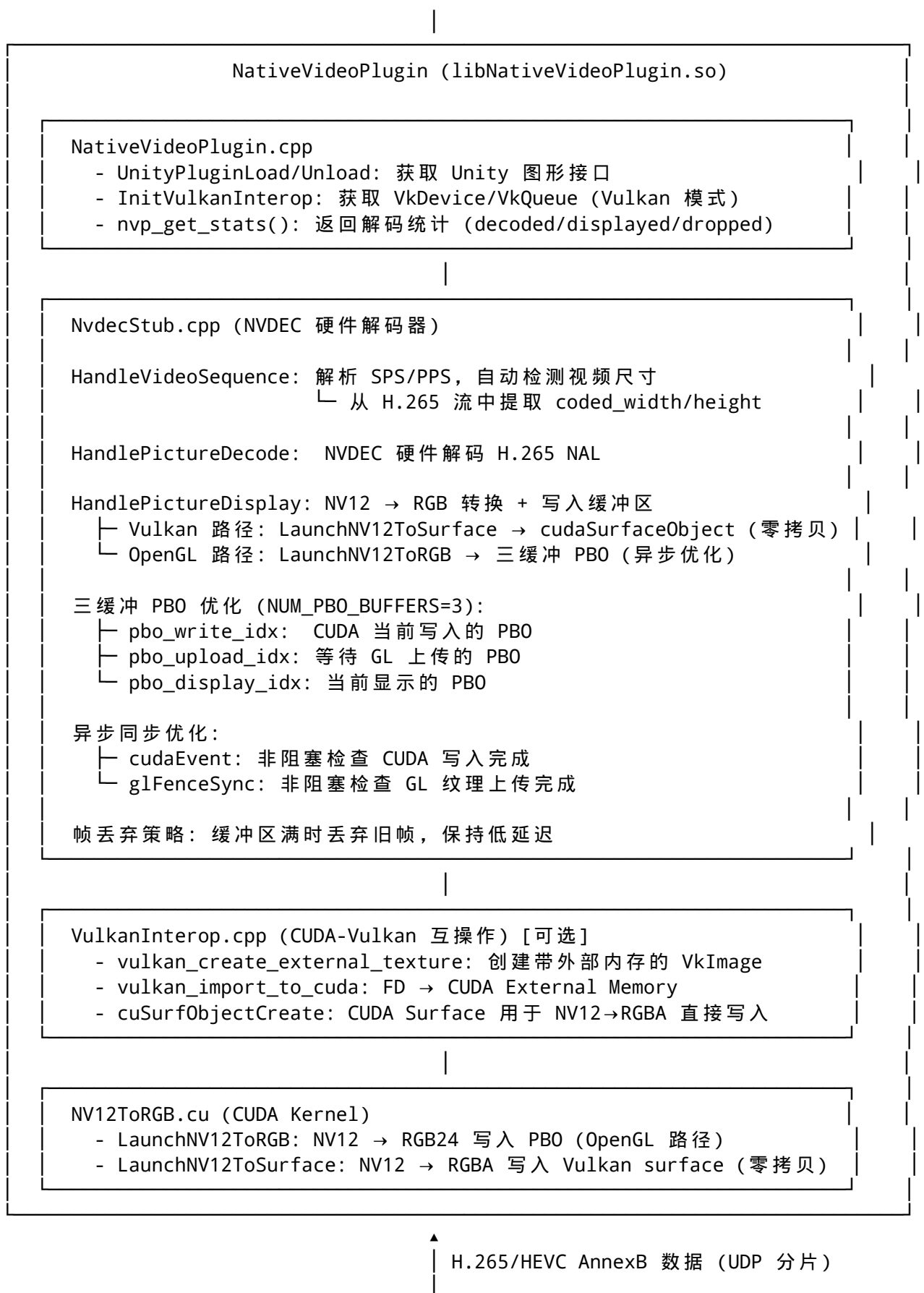
10. 启动流程

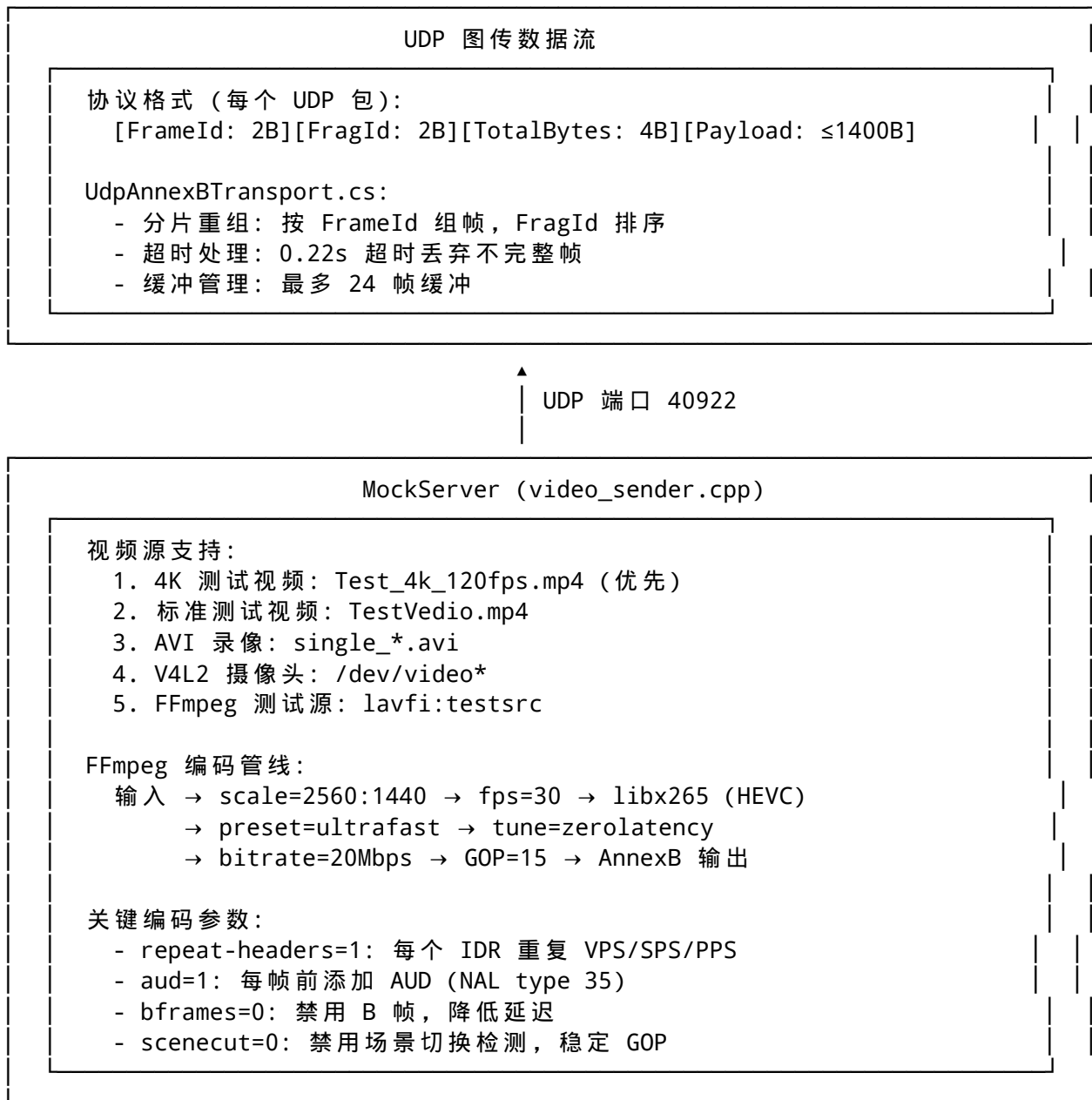
11. 故障排查

12. 版本历史

1. 系统概览







2. 硬件自适应系统



- SystemInfo.graphicsDeviceVendor → 厂商识别
- SystemInfo.graphicsMemorySize → 显存大小

2. CPU/内存信息

- SystemInfo.processorCount → CPU 核心数
- SystemInfo.systemMemorySize → 系统内存

3. 平台特定检测

- Linux: 检查 /dev/dri/renderD128 (VA-API 可用性)



GPU 厂商识别 (GpuVendor)

Nvidia	"nvidia", "geforce", "quadro", "rtx", "gtx"
Intel	"intel", "iris", "uhd graphics", "hd graphics"
Amd	"amd", "radeon", "vega"
Apple	"apple", "m1", "m2", "m3"



能力等级判定 (CapabilityLevel)

High (高配):

- NVIDIA 独显 + VRAM ≥ 4GB
- AMD 独显 + VRAM ≥ 4GB

Mid (中配):

- NVIDIA 低端独显 (< 4GB VRAM)
- AMD 独显 (< 4GB VRAM)
- Intel Iris/Xe 集显 + CPU ≥ 8核
- Apple Silicon (M1/M2/M3)

Low (低配):

- Intel 集显 + CPU ≤ 4核 (典型: i3 + UHD)
- 系统内存 < 8GB



推荐加速模式 (RecommendedAccel)

平台	GPU	推荐加速
----	-----	------

Linux	NVIDIA 独显	NvdecCuda
Linux	Intel/AMD	Vaapi
Windows	NVIDIA 独显	NvdecCuda
Windows	Intel/AMD	Dxva (D3D11VA)
macOS	Any	VideoToolbox
其他/不支持	-	Software (软解)



推荐配置输出

等级	分辨率	目标帧率	队列大小	每帧消费数
Low	1280×720	30 fps	4	1
Mid	1920×1080	60 fps	6	2
High	1920×1080	120 fps	8	3

3. 解码后端架构

VideoStreamService.cs 解码后端自动选择

Awake() 阶段：

1. 调用 HardwareCapabilityDetector.Detect() 获取硬件信息
2. 检查配置的 decodeBackend:
 - └ 若设置为 NativeNvdec 但硬件非 NVIDIA → 自动降级为 FfmpegPipe
 - └ 若设置为 FfmpegPipe → 直接使用 ffmpeg 管道
3. 根据 ConfigLoader 配置设置 maxApplyFps (纹理上传频率上限)

解码后端对比

路径 A	NativeNvdec (原生插件)
仅 NVIDIA	• NVIDIA NVDEC 硬件解码 • CUDA → OpenGL/Vulkan 零拷贝 • 三缓冲 PBO 异步优化 • 延迟最低，性能最优

路径 B	FfmpegPipeDecoder (管道解码)
跨平台	- 支持多平台硬件加速: - CUDA (NVIDIA) - VAAPI (Intel/AMD Linux) - D3D11VA/DXVA (Windows) - VideoToolbox (macOS) - Software (纯软解回退) - 启动时自动检测最优加速模式 - 硬件不可用时自动回退软解

FFmpeg 硬件加速命令行示例

NVIDIA CUDA/NVDEC (Linux/Windows)

```
ffmpeg -hwaccel cuda -hwaccel_output_format cuda -extra_hw_frames 8 \
-f hevc -i - -vf scale_cuda=1920:1080:format=nv12,hwdownload,format=nv12,format=rgb24
-pix_fmt rgb24 -f rawvideo pipe:1
```

Intel/AMD VAAPI (Linux)

```
ffmpeg -hwaccel vaapi -hwaccel_output_format vaapi -vaapi_device /dev/dri/renderD128 \
-f hevc -i - -vf scale_vaapi=1920:1080:format=nv12,hwdownload,format=nv12,format=rgb24
-pix_fmt rgb24 -f rawvideo pipe:1
```

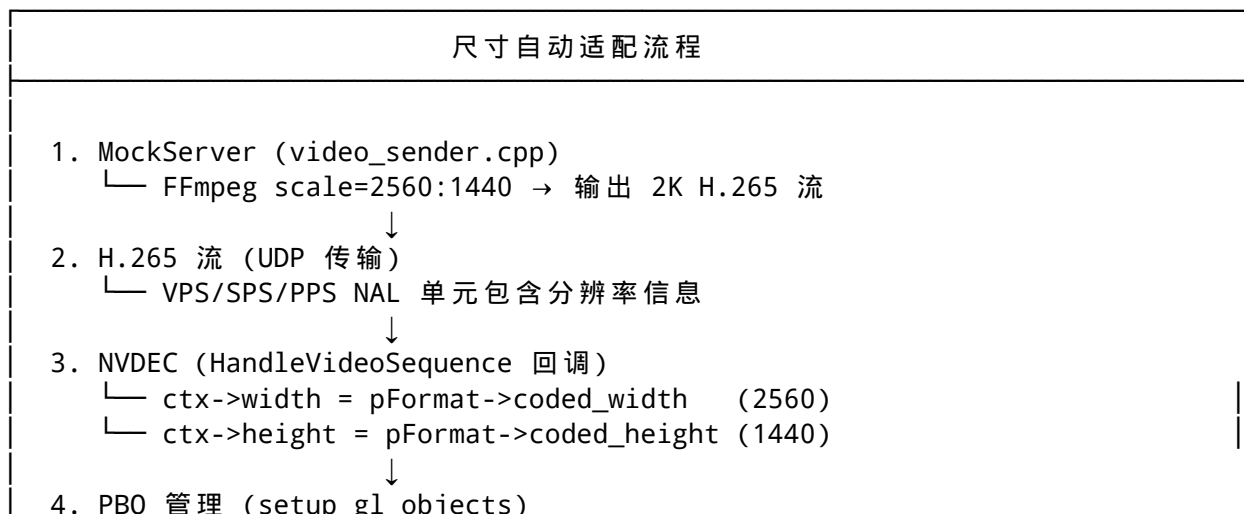
Windows D3D11VA

```
ffmpeg -hwaccel d3d11va -f hevc -i - \
-vf scale=1920:1080,format=rgb24 -pix_fmt rgb24 -f rawvideo pipe:1
```

macOS VideoToolbox

```
ffmpeg -hwaccel videotoolbox -f hevc -i - \
-vf scale=1920:1080,format=rgb24 -pix_fmt rgb24 -f rawvideo pipe:1
```

4. 图传尺寸控制链路



- └ 预分配 4K 尺寸: MAX_WIDTH=3840, MAX_HEIGHT=2160
- └ 实际显示尺寸: gl_width/gl_height = ctx->width/height
- └ 三缓冲 PBO: 每个 ~25MB, 总计 ~75MB GPU 内存



5. NativeVideoPlugin (nvp_get_stats)
└ 返回 { width, height, decoded, displayed, dropped }



6. Unity (VideoStreamService.Update)
└ 检测 stats.width != nativeTexWidth?
→ 是: 销毁旧纹理, 创建新尺寸的 Texture2D
→ 否: 继续使用当前纹理

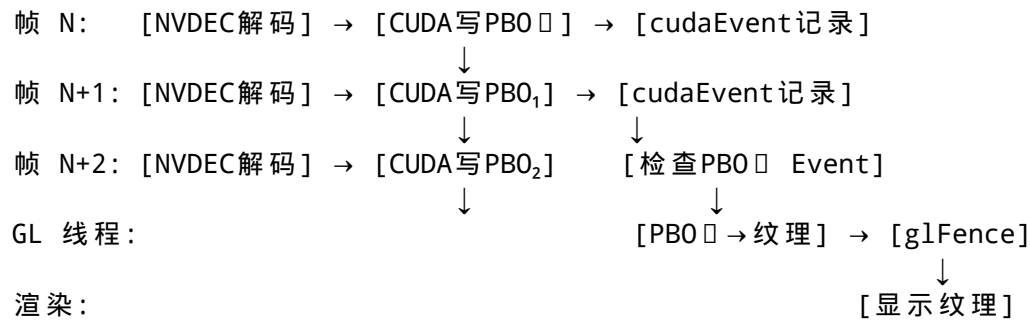


7. RawImage (MainScene.unity)
└ RectTransform Anchor (0,0)-(1,1) 全屏自适应拉伸

5. 三缓冲 PBO 工作流程 (OpenGL 路径)

三缓冲 PBO 异步流水线

时间轴 →



缓冲区状态:

pbo_write_idx = 当前 CUDA 写入位置 (循环: 0→1→2→0)
 pbo_upload_idx = 当前 GL 上传位置
 pbo_cuda_pending[i] = PBO[i] 是否有未完成的 CUDA 写入
 pbo_ready_for_upload[i] = PBO[i] 是否已准备好上传

帧丢弃策略 (缓冲区满时):

```

if (pbo_ready_for_upload[write_idx] && cudaEvent not ready):
    frames_dropped++; return; // 丢弃当前帧, 保持低延迟
  
```

6. 数据结构定义 (NvdecStub.h)

```
struct NvdecContext {
    // 4K 120fps 优化常量
    static constexpr int NUM_PBO_BUFFERS = 3;    // 三缓冲 PBO
    static constexpr int MAX_WIDTH = 3840;       // 4K 宽度预分配
    static constexpr int MAX_HEIGHT = 2160;      // 4K 高度预分配
    static constexpr size_t MAX_PBO_SIZE = MAX_WIDTH * MAX_HEIGHT * 3; // ~25MB

    // CUDA 上下文
    CUcontext cuCtx;
    CUvideoctxlock cuLock;
    CUvideodecoder decoder;
    CUvideoparser parser;
    cudaStream_t stream;

    // 视频尺寸 (从流中自动检测)
    int width, height;

    // OpenGL 三缓冲 PBO
    unsigned int pbo[3];           // GL Buffer IDs
    unsigned int tex;              // GL Texture ID
    CUgraphicsResource cuPbo[3];   // CUDA-GL 互操作资源
    int pbo_write_idx;             // CUDA 写入索引
    int pbo_upload_idx;           // GL 上传索引
    int gl_width, gl_height;       // 当前 GL 对象尺寸

    // 异步同步
    cudaEvent_t cudaWriteEvent[3]; // CUDA 写入完成事件
    bool pbo_cuda_pending[3];      // CUDA 写入进行中
    bool pbo_ready_for_upload[3];  // 准备好 GL 上传
    void* glFence;                 // GLsync fence

    // 统计
    std::atomic<int> frames_decoded;
    std::atomic<int> frames_displayed;
    std::atomic<int> frames_dropped; // 因缓冲区满丢弃的帧

    // Vulkan 路径 (可选)
    VulkanInteropContext vkCtx;
    bool use_vulkan;
    cudaSurfaceObject_t vkSurface;
};
```

7. 配置分级系统

ConfigLoader.cs 自动配置档位选择
启动流程： 1. HardwareCapabilityDetector.Detect() → 获取 CapabilityLevel 2. 根据等级选择配置文件：

- └─ Low → Config/params_lowspec.json
 - └─ Mid → Config/params_midspec.json
 - └─ High → Config/params.json
3. 配置文件不存在时自动回退到 params.json
4. 应用硬件推荐配置补充缺失字段

配置文件对比

配置项	params_lowspec.json	params_midspec.json	params.json (高配)
decoderOutputWidth	1280	1920	1920
decoderOutputHeight	720	1080	1080
targetFrameRate	30	60	120
decoderQueueSize	4	6	8
maxDrainPerUpdate	1	2	2
logBufferSize	8	8	4
maxFileQueueSize	32	48	64

8. 性能指标

指标	720p (低配)	1080p (中配)	2K (2560×1440)	4K (3840×2160)
目标帧率	30 fps	60 fps	30-60 fps	30-120 fps
单帧 RGB 大小	~2.8 MB	~6.2 MB	~11 MB	~25 MB
三缓冲 PBO 总内存	~8 MB	~19 MB	~33 MB	~75 MB
NVDEC 解码表面	4-6 个	6-8 个	8-12 个	8-12 个
总 GPU 内存占用	~30 MB	~60 MB	~100 MB	~275 MB
延迟 (端到端)	<80ms	<60ms	<50ms	<80ms

9. 关键文件清单

核心服务层

文件	路径	功能
VideoStreamService.cs	Assets/Scripts/Framework/Video/	Unity 端视频服务, 纹理管理
FfmpegPipeDecoder.cs	Assets/Scripts/Framework/Video/	FFmpeg 管道解码器 (跨平台)
NativeVideoBridge.cs	Assets/Scripts/Framework/Video/	C# ↔ Native 桥接
HardwareCapabilityDetector.cs	Assets/Scripts/Framework/Boot/	硬件能力探测与自适应
ConfigLoader.cs	Assets/Utils/	配置加载与分档选择
RuntimeTuner.cs	Assets/Scripts/Framework/Boot/	运行时性能调优

原生插件层

文件	路径	功能
NvdecStub.h/cpp	Assets/Plugins/NativeVideoPlugin/	NVDEC 解码器 + PBO 管理

文件	路径	功能
NV12ToRGB.cu VulkanInterop.cpp	Assets/Plugins/NativeVideoPlugin/ Assets/Plugins/NativeVideoPlugin/	CUDA 色彩空间转换内核 Vulkan 互操作 (可选)

配置文件

文件	路径	功能
params.json params_midspec.json params_lowspec.json	Assets/StreamingAssets/Config/ Assets/StreamingAssets/Config/ Assets/StreamingAssets/Config/	高配参数 (1080p 120fps) 中配参数 (1080p 60fps) 低配参数 (720p 30fps)

测试工具

文件	路径	功能
video_sender.cpp start_mock.sh check_nvidia.sh	Assets/MockServerCpp/src/ Assets/MockServerCpp/ (项目根目录)	MockServer 视频发送 MockServer 启动脚本 NVIDIA 环境自检脚本

10. 启动流程

```
# 1. 环境检查 (可选)
./check_nvidia.sh                # 检查 NVIDIA 驱动、CUDA、ffmpeg 环境

# 2. 启动 MockServer (视频源)
cd Assets/MockServerCpp
./start_mock.sh                  # 自动选择 Test_4k_120fps.mp4

# 3. 启动 Unity (OpenGL 模式)
./RoboMasterClientWMJ -force-opengl

# 4. 监控日志
tail -f Log/DebugLog.txt | grep -E "NVDEC|VideoStream|Hardware"

# 查看硬件检测结果
grep "HardwareDetection" Log/DebugLog.txt
```

11. 故障排查

问题	可能原因	解决方案
黑屏无图像	MockServer 未启动	./start_mock.sh
画面撕裂	双缓冲不足	已升级为三缓冲
帧率低	GPU 内存不足	检查 nvidia-smi
尺寸错误	PBO 未重建	会自动检测并重建
CUDA-GL 失败	GPU 不匹配	确保 Unity 使用 NVIDIA GPU
硬件检测等级错误	SystemInfo 返回异常	检查日志中的 HardwareDetection 输出
ffmpeg 硬解失败	缺少硬件加速驱动	运行 ./check_nvidia.sh 诊断

问题	可能原因	解决方案
低配机器卡顿 VA-API 不可用	配置档位未正确选择 缺少 renderD128 设备	检查是否加载了 params_lowspec.json 安装 intel-media-driver/mesa-va-drivers

12. 版本历史

- v1.0: 初始双缓冲 PBO 实现
- v1.1: 添加 Vulkan 路径支持
- v1.2: 修复视频尺寸检测
- v2.0: 三缓冲 PBO + 异步 CUDA Event + GL Fence 优化
- v2.1: 4K 预分配 + 帧丢弃策略
- v3.0: 硬件自适应系统 (当前版本)
 - 新增 HardwareCapabilityDetector 硬件能力探测
 - 支持 Low/Mid/High 三级配置自动选择
 - FfmpegPipeDecoder 支持 VA-API/DXVA/VideoToolbox 跨平台硬解
 - VideoStreamService 自动降级非 NVIDIA 硬件到 ffmpeg 后端
 - 新增低配/中配专用参数文件